

Consequentialism: Counterfactual Sampling to Speed Learning

1st Adrian Manchado

*Diercks School of Advanced Computing
Milwaukee School of Engineering
Milwaukee, WI USA
BLARBLAR@msoe.edu*

2nd Jeremy Kedziora

*Diercks School of Advanced Computing
Milwaukee School of Engineering
Milwaukee, WI USA
kedziora@msoe.edu*

Abstract—Experience replay prioritization in deep reinforcement learning has focused on *transition surprise* — measured by temporal-difference (TD) error — yet the most consequential decisions often produce low TD error once the value function has locally converged. We introduce *Counterfactual Consequence Estimation* (CCE), a method that scores each replay transition by the total-variation divergence between the return distribution realized under the taken action and those achievable under counterfactual alternatives, estimated via Monte Carlo rollouts of the current policy. CCE priority is mixed with TD-error priority either additively or multiplicatively, targeting transitions that are both consequential and poorly understood by the value function. Across three environments evaluated with the reliable statistical framework (10 seeds each), we find: (1) CCE scores correlate strongly with exact oracle importance on FrozenLake ($\rho = 0.889 \pm 0.031$ at convergence); (2) multiplicative CCE+TD mixing reaches IQM ≈ 1.0 on deterministic FrozenLake versus ≈ 0.45 for DQN+PER; (3) no benefit appears in the stochastic variant, as expected when environment noise dilutes the consequence signal; and (4) multiplicative mixing achieves a modest gain in SMAX 3m (IQM 0.72 vs. 0.68 for DQN+PER).

Index Terms—reinforcement learning, experience replay, prioritization, counterfactual reasoning, sample efficiency

I. INTRODUCTION

Reinforcement learning (RL) agents learn by interacting with an environment and updating from the resulting experience. In problems with sparse rewards, long horizons, or high-dimensional state spaces, generating enough experience to learn a competent policy is expensive — often requiring millions of environment interactions. Experience replay [10] partially addresses this by storing transitions in a buffer and reusing each one across multiple gradient updates. Prioritized Experience Replay (PER) [12] refines this further by concentrating sampling on transitions with high temporal-difference (TD) error, the agent’s surprise, yielding substantial gains in sample efficiency.

TD error, however, measures whether a transition was *surprising* to the value function — not whether it was *consequential* to the episode outcome. A transition may produce large TD error simply because the Q-network is poorly initialized in that region, regardless of whether the action taken actually mattered. Conversely, a pivotal decision — one where the chosen action substantially changed the distribution of future returns relative to alternatives — may produce small TD

error once the value function has locally converged, even if that decision still determines whether the episode is won or lost. These high-consequence, low-surprise transitions are systematically undersampled by PER.

We propose *Counterfactual Consequence Estimation* (CCE), a method that scores transitions by asking: how much would the return distribution have changed if a different action had been taken? For each stored transition, CCE rolls out n counterfactual trajectories under each alternative action using the current policy, estimates the resulting return distributions, and measures their divergence from the realized distribution. Transitions where the action choice substantially shifted outcome distributions receive high CCE priority. This score is then mixed with standard TD-error priority to form a balanced replay distribution that simultaneously targets surprising and consequential transitions.

This paper makes three contributions. First, we introduce CCE (Algorithm 1), a Monte Carlo estimator of transition consequence that is applicable to any replay-based RL algorithm. Second, we propose DQN with CCE-augmented priority (Algorithm 2), with both additive and multiplicative mixing schemes that interpolate between TD-error and CCE priority. Third, we provide an empirical evaluation across three environments — FrozenLake 8×8 (deterministic and stochastic variants) and SMAX 3m — with 10 independent seeds and the reliable statistical framework [1], demonstrating that CCE leads to more sample-efficient early training while preserving asymptotic performance.

II. RELATED WORK

Prioritized Replay. Deep Q-Networks (DQN) [10] established the experience-replay framework: transitions $(s_t, a_t, r_{t+1}, s_{t+1})$ are stored in a circular buffer and sampled uniformly for stochastic gradient updates to the Q-network. Prioritized Experience Replay (PER) [12] replaced uniform sampling with a distribution weighted by TD error, showing substantial gains in sample efficiency; PER is the direct ancestor of the priority-mixing scheme presented in this work. Large Batch Experience Replay (LaBER) [8] frames replay sampling as importance sampling for gradient estimation, derives the theoretically optimal sampling distribution, and approximates it by drawing a

large candidate batch and retaining the highest-weight subset; their published results across Atari games provide a natural baseline for replay-prioritization comparisons.

Critical and Consequential States. A recurring observation in the RL literature is that episode outcomes are often determined by a small number of key decision points. [6] define *critical states* as those in which a policy strongly prefers a narrow subset of actions and demonstrate that surfacing these states to human supervisors improves trust calibration and intervention timing. [7] identify critical states via the variance of the Q-function across actions and concentrate exploration on them; CCE can be viewed as a distributional generalization of this scalar variance signal, replacing Q-value variance with a divergence over full return distributions under counterfactual actions. [5] define *true criticality* as the expected reward drop when an agent executes n consecutive random deviations from its policy, then validate proxy criticality metrics against this ground truth; their evaluation methodology provides the closest published framework for assessing priority signals of the kind CCE produces. [9] train a return-prediction model on video-encoded episodes and apply mask-based sensitivity to localize critical frames (Deep State Identifier), explicitly adopting the same few-states-matter framing as this work; unlike CCE, their method operates offline on pre-collected visual data rather than online on raw state-action pairs.

Adjacent Work. RUDDER [2] addresses the related but distinct problem of credit assignment under delayed rewards by redistributing the episode return to individual transitions; unlike CCE, RUDDER modifies the effective reward signal and Bellman backup, whereas CCE leaves both unchanged and acts solely on the replay sampling distribution. CF-GPS [4] applies structural causal models to synthesize alternative episode trajectories under counterfactual actions as additional training targets for policy search in POMDPs; CF-GPS uses counterfactuals to generate new training data, while CCE uses them to prioritize which observed transitions to replay.

III. BACKGROUND

In reinforcement learning, control problems are modeled as a Markov Decision Process (MDP) [3] which is defined by:

- A set of states S that describe the current environmental conditions facing the agent;
- A set of actions $A(s)$ that an agent can take;
- Probabilities $p(s' | a, s)$ for transitioning from state s to state s' given action a ;
- A function $r : S \times A \times S \rightarrow \mathbb{R}$ so that $r(s', a, s)$ supplies the immediate reward associated with this transition.

In environments that take place across a finite number of discrete periods T , the sequence of periods the agent participates in is referred to as an episode. The goal of the agent is to learn a policy $\pi(a|s)$, which describes the probability that a trained agent should take action a in state s , to maximize the sequence of rewards across an episode: $\sum_{t=0}^T \gamma^t r(s_{t+1}, a_t, s_t)$. Here a_t and s_t are the action and state at time t and $\gamma \in [0, 1]$ is the discount factor on future rewards. Throughout we will slightly abuse notation to write $r(s_{t+1}, a_t, s_t)$ as r_{t+1} .

A. Trajectories and Returns

The choices of an agent via its policy $\pi(\cdot)$ throughout the course of an episode lead to a realized time series of information commonly referred to as a trajectory:

$$\tau_\pi = s_0, a_0, r_1, s_1, a_1, \dots, r_{T-1}, s_{T-1}, a_{T-1}, r_T, s_T.$$

In general, it may be useful to refer to partial trajectories below. We will denote a slice of a trajectory generated by policy $\pi(\cdot)$, beginning at time t , and ending at time t' as $\tau_\pi^{(t:t')}$. Thus $\tau_\pi^{(0:t)}$ will refer to the trajectory data up until time t and $\tau_\pi^{(t:T)}$ will refer to the trajectory data from t to the end of the episode. The return at time t associated with a slice from a trajectory of data is the sum of discounted rewards within the trajectory:

$$G(\tau_\pi^{(t:t')}) = \sum_{j=t+1}^{t'} \gamma^{j-t-1} r_j.$$

Given this, for any specific time point t , the remaining return for an episode generated by $\pi(\cdot)$ can be written as $G(\tau_\pi^{(t:T)})$.

B. Measuring Important Moments

One way to conceptualize the measurement of key moments is to take a counterfactual approach and ask “what would have happened if” questions of the policy and the data generated by it. Given a trajectory τ_π at any moment t the agent took an historic action a_t according to the distribution associated with $\pi(s_t)$ leading to the realized trajectory with probability:

$$p(s_T | s_{T-1}, a_{T-1}) \prod_{j=t}^{T-1} p(s_{j+1} | s_j, a_j) \pi(a_{j+1} | s_{j+1})$$

which depends on both the policy and unobserved dynamics of the MDP. Thus, at time t , from the perspective of the agent $\tau_\pi^{(t:)} \sim d_{\tau_\pi^{(t:)}}(s_t, a_t)$, where $d_{\tau_\pi^{(t:)}}(\cdot)$ is a distribution with dependence on the action and state. Since the remaining return is entirely determined by the remaining trajectory $G(\tau_\pi^{(t:)})$ also follows $d_{\tau_\pi^{(t:)}}(\cdot)$. Formally, let:

$$\mathcal{R}(g) = \{\tau^t : G(\tau^t) = g\}$$

be the set of all trajectories beginning at time t whose return will equal g . Then:

$$d_G^{(\pi)}(g | s_t, a) \equiv \int_{\mathcal{R}(g)} d_{\tau_\pi^{(t:)}}(\tau | s_t, a) d\tau. \quad (1)$$

That is, the distribution over future returns $G(\tau_\pi^{(t:)})$ is derived from “adding up” the trajectories that lead to a given return.

Consider an intervention – an enforced change at time t from the historic action a_t to a feasible, alternative action $a \in A(s_t)$. Such an intervention leads to a natural comparison of two distributions, the one for the returns derived from historic action $d_G^{(\pi)}(g | s_t, a_t)$ and the one for the unrealized trajectories that would have followed the alternative action a , $d_G^{(\pi)}(g | s_t, a)$; here we consider the use of a generic metric,

$m(\cdot)$, that quantifies the difference amongst a set of distributions $d_G^{(\pi)}(g|s_t, a_1), \dots, d_G^{(\pi)}(g|s_t, a_{|A|})$.¹

Algorithm 1 Counterfactual Consequence Estimation

Input: policy π , state-action pair (s_t, a_t) , and $n \in \mathbb{N}_+$

- 1: **for** $a \in A(s_t)$ **do**
- 2: Use π to sample n trajectories $\tau_{\pi,a}^{j,t} \sim d_{\tau_{\pi,a}^{(t)}}(s_t, a)$
- 3: For each sampled trajectory j compute $G(\tau_{\pi,a}^{j,t})$
- 4: Estimate $d_G^{(\pi)}(s_t, a)$ from $G(\tau_{\pi,a}^{1,t}), \dots, G(\tau_{\pi,a}^{n,t})$
- 5: **end for**
- 6: Compute $m(\{d_G^{(\pi)}(s_t, a)\}_{a \in A})$

IV. CONSEQUENCE PRIORITIZATION

One straightforward way to apply the sampling-based method of algorithm 1 is to use it as a way to adjust how the agent encounters data to learn from. For example, [12] extended the DQN algorithm introduced by [10] with prioritized experience replay, in which the j th past transition from the replay buffer is sampled according to:

$$p^\delta(j) = \frac{(m_j^\delta + \epsilon)^\beta}{\sum_{i=1}^{|D|} (m_i^\delta + \epsilon)^\beta} \quad (2)$$

where $m_j^\delta = |\delta_j|$ is most recent temporal difference error associated with j , and ϵ ensures that each transition always has a positive probability of being sampled and β controls the entropy.

We experiment with augmenting TD priority with measurements of consequence. Given counterfactual consequence estimates from algorithm 1, in analogue with 2, we could compute:

$$p^C(j) = \frac{(m_j^C + \epsilon)^\beta}{\sum_{i=1}^{|D|} (m_i^C + \epsilon)^\beta} \quad (3)$$

where $m_j^C = m(\{d_G^{(\pi)}(s_j, a)\}_{a \in A})$, and set the overall priority as:

$$p(j) = \frac{\mu p^C(j) + (1 - \mu) p^\delta(j)}{\sum_{k=1}^{|D|} \mu p^C(k) + (1 - \mu) p^\delta(k)} \quad (4)$$

$$p(j) = \frac{p^C(j)^{\mu_C} p^\delta(j)^{\mu_\delta}}{\sum_{k=1}^{|D|} p^C(k)^{\mu_C} p^\delta(k)^{\mu_\delta}} \quad (5)$$

where μ controls the relative contribution of consequence estimates to sampling probabilities. This balanced approach puts the highest priority on transitions that are both important (high consequence) and poorly modeled (high TD error).

¹There are many options to compare distributions. For example, if we used KL-divergence we could set:

$$m(a_t, a) = \int_{-\infty}^{\infty} d_G^{(\pi)}(g|s_t, a_t) \ln \left\{ \frac{d_G^{(\pi)}(g|s_t, a_t)}{d_G^{(\pi)}(g|s_t, a)} \right\} dg.$$

We consider multiple metrics in our experiments below.

Algorithm 2 DQN with balanced consequence-error priority

Input: $\mu, \gamma, \alpha, \epsilon, \beta, M, B_{est}^C, B_{up}, K_{up}, K_{tar}$

- 1: **Init:** Q network parameters $\mathbf{w}$, replay buffer $D = \emptyset$
- 2: Set $\mathbf{w}' = \mathbf{w}$ and $\pi(s)$
- 3: Sample $s_0 \in S$
- 4: **for** $t = 1, 2, 3, \dots$ **do**
- 5: Sample a_t according to $\pi(s_t)$, observe r_{t+1} and s_{t+1}
- 6: Add $(s_t, a_t, r_{t+1}, s_{t+1})$ to D
- 7: Set $m_t^C = \sum_{j=1}^{|D|} m_j / |D|$
- 8: Set $m_t^\delta = \max_{j=1, \dots, |D|} \{m_j\}$
- 9: **if** $|D| > M$ **then** drop the oldest
- 10: **if** $t \bmod K_{up} = 0$ **then**
- 11: Sample $(s_j, a_j, r_j, s'_j)_{j=1}^{B_{est}^C}$ from $U(D)$
- 12: Update m_j^C via algorithm 1 and then update $p(j)$
- 13: Sample $(s_j, a_j, r_j, s'_j)_{j=1}^{B_{up}}$ from D via $p(j)$
- 14: Compute importance sampling weights so that:
$$w_j = (p(j)|D|)^{-1}$$
- 15: Compute TD error:
$$\delta_j \leftarrow r_j + \gamma \max_a \{Q_\pi(s'_j, a|\mathbf{w}')\} - Q(s_j, a_j|\mathbf{w})$$
- 16: Update TD priorities: $m_j^\delta \leftarrow |\delta_j|$
- 17: Update Q network weights:
$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla_{\mathbf{w}} \left(\frac{1}{B_L} \sum_{j=1}^{B_L} w_j \delta_j^2 \right)$$
- 18: **end if**
- 19: **if** $t \bmod K_{tar} = 0$ **then** $\mathbf{w}' \leftarrow \mathbf{w}$
- 20: **end for**

TABLE I: Hyperparameters

Hyperparameter	Space	Description
μ	$[0, 1]$	Weight on consequence metric
γ	$[0, 1]$	Discount factor
α	$(0, 1]$	Step size
ϵ	$\mathbb{R}_{\geq 0}$	Priority shaping parameter
β	$\mathbb{R}_{\geq 0}$	Priority shaping parameter
B_{est}^C	$\mathbb{N}_1$	Batch size for consequence estimates
B_{up}	$\mathbb{N}_1$	Batch size for Q network update
M	$\mathbb{N}_1$	Replay buffer memory size
K_{up}	$\mathbb{N}_1$	Frequency of Q network updates
K_{tar}	$\mathbb{N}_1$	Frequency of target network updates

V. EXPERIMENTS

A. Environments

We evaluate CCE across three environments spanning stochastic single-agent and multi-agent settings, chosen to stress-test the priority signal under different levels of environment noise, state spaces, and action structures.

SMAX 3m. The StarCraft Multi-Agent Challenge, re-implemented in JAX via JaxMARL [11]. Three allied marines face three enemies in a combat scenario with shaped reward: agents receive a per-step signal proportional to the fraction of enemy health destroyed, plus a +10 bonus upon winning

the battle. All agents share a single centralized replay buffer; the global state fed to the Q-network is the concatenation of per-agent observations. Evaluation runs a greedy policy against the built-in heuristic opponent (`HeuristicEnemySMAx`) for 100 episodes.

FrozenLake 8×8 (Deterministic). An 8×8 FrozenLake environment re-implemented in JAX for GPU execution, with the slippery flag disabled. Every action moves the agent deterministically, isolating the policy-driven consequence signal from stochastic outcome noise. Reward is +1 on reaching the goal and 0 otherwise.

FrozenLake 8×8 (Stochastic). The same map with stochastic ice transitions: each action has a $\frac{1}{3}$ probability of sliding perpendicular to the intended direction. This injects outcome variance that cannot be attributed to the agent’s action choice, providing an expected null condition for CCE — a controlled test of whether CCE hurts when its signal is diluted.

B. Algorithms

We compare five configurations across all environments and seeds.

TABLE II: Algorithms compared in Claim 2

Label	Configuration
DQN-Uniform	DQN, uniform replay
DQN+PER	DQN, TD-error priority (Eq. 2)
DQN+CCE-only	CCE priority only; $\mu = 1$ in Eq. 5
CCE+TD (add)	Additive mixing (Eq. 5), μ from sweep
CCE+TD (mul)	Multiplicative mixing, $\mu_C = \mu_\delta = 1$

DQN-Uniform provides a no-prioritization floor; DQN+PER is the primary baseline against which CCE improvements are measured. DQN+CCE-only isolates the CCE signal from TD error. The two mixed variants test whether combining both signals improves over either alone and which mixing scheme is preferable.

C. Claim 1: CCE Identifies Consequential Moments

To validate that CCE scores correlate with ground-truth transition importance, we compare CCE priority scores against oracle assessments obtained independently of the learning algorithm.

FrozenLake exact oracle. FrozenLake’s stochastic dynamics are fully specified by a precomputable transition table $P[s][a]$ — for the 8×8 slippery map, each action produces three equally probable outcomes corresponding to the intended direction and its two perpendicular neighbours. This known structure enables *exact* ground-truth labeling via value iteration: running Bellman backups on P to convergence yields $Q^*(s, a)$ for all state-action pairs without any learned approximation. We define the oracle importance of state s as the average expected-return cost of acting suboptimally:

$$\text{Oracle}(s) = \frac{1}{|A(s)| - 1} \sum_{a \neq a^*} [Q^*(s, a^*) - Q^*(s, a)],$$

$$a^* = \underset{a'}{\operatorname{argmax}} Q^*(s, a')$$
(6)

the mean suboptimality gap across all non-optimal actions. A high oracle score indicates that acting suboptimally at s is costly — the state is genuinely consequential. We compute CCE scores for all 53 non-terminal states and report the Spearman rank correlation ρ between CCE scores and $\text{Oracle}(s)$ at three training stages: untrained (ep. 150), mid-training (ep. 3,900), and fully trained (best checkpoint).

D. Claim 2: Sample Efficiency

We evaluate whether CCE leads to faster early learning using the reliable evaluation framework [1], following recommended practices for statistically robust deep RL comparisons.

Hyperparameter selection. CCE introduces two key choices: the divergence metric $m(\cdot)$ and the mixing weight μ . To select these without contaminating main results, we ran a two-phase sweep on SMAx 3m using 3 seeds per configuration, with all sweep seeds held out from the main experiment. *Phase 1 (metric sweep):* four divergence metrics (Wasserstein, KL divergence, Jensen-Shannon, total variation) $\times$ 3 seeds = 12 runs, with $\mu = 0.5$ fixed. Total variation achieved the highest mean win rate and was selected. *Phase 2 (μ sweep):* four values $\mu \in \{0.25, 0.5, 0.75, 1.0\}$ $\times$ 3 seeds = 12 runs using total variation. $\mu = 0.25$ achieved the highest mean win rate and was selected. The pair (total variation, $\mu=0.25$) is applied without modification to all three environments.

Main experiment. All five algorithms run for 10 independent seeds on each of the three environments (150 runs total). We report three metrics:

- 1) *IQM learning curves.* Interquartile mean win rate at each evaluation checkpoint with 95% stratified bootstrap confidence intervals [1]. One curve per algorithm per environment; the IQM trims the two or three most extreme seeds before averaging, making the curve robust to outliers.
- 2) *Final IQM.* IQM win rate averaged over the final 10% of training checkpoints per seed. Tests whether early efficiency gains come at the cost of asymptotic performance.
- 3) *P(improvement).* For each CCE variant, the probability that a randomly drawn seed beats DQN+PER on final win rate, estimated via stratified bootstrap. Reported per environment with 95% confidence intervals; with 10 seeds the intervals span roughly ± 10 –15 percentage points, so we present this as directional evidence rather than a tight statistical claim.

E. Implementation Details

All Q-networks are multi-layer perceptrons with ReLU activations. Table III summarizes the key hyperparameters per environment; values shared across all environments are listed once.

For all environments, ε decays linearly from 1.0 to 0.05 over the stated number of episodes; PER uses $\epsilon_{\text{PER}} = 0.01$ and $\beta = 0.25$. All experiments run on NVIDIA Tesla T4 GPUs on the MSOE Rosie high-performance computing cluster.

TABLE III: Implementation hyperparameters per environment

	SMAX 3m	FrozenLake 8×8
Hidden dim	128 (2 layers, 1 head)	64 (2 layers)
Layer norm	No	No
γ	0.95	0.99
α	5e-4	1e-3
Replay M	100k	100k
Batch B	32	32
Target update C	500 steps	200 steps
Q-update freq	4 steps	every step
ϵ decay	10k ep	10k ep
CF horizon H	30	10
CF rollouts n	30	20
Score interval	200 ep	100 ep
Score sample B^C	256	128

For CCE, the counterfactual discount γ_{CF} matches the training discount. The mixing weight μ and divergence metric are selected by the two-phase sweep described in Section 5.4; the CCE-only variant fixes $\mu = 1$ and the multiplicative variant uses $\mu_C = \mu_\delta = 1$.

VI. RESULTS

We organize results by claim. All confidence intervals are 95% stratified bootstrap (10,000 resamples).

A. Claim 1 — CCE Identifies Consequential Moments

Claim 1 asks whether CCE scores, computed purely from rollouts under the current policy, correlate with ground-truth state importance. The oracle (Eq. 6) is derived from dynamic programming on FrozenLake’s known transition table; it has no access to CCE rollouts. CCE has no access to Q^* . These two signals are completely independent.

Figure 1 shows the CCE-vs-oracle scatter at three training stages for seed 0. At episode 150 the agent is effectively untrained and the correlation is moderate ($\rho = 0.319 \pm 0.114$); one seed is not yet significant ($p = 0.258$). By mid-training (ep. 3,900) the policy has converged enough for CCE rollouts to yield informative return distributions, and ρ jumps to 0.765 ± 0.096 (all seeds $p < 0.001$). At the best checkpoint, the correlation reaches $\rho = 0.889 \pm 0.031$ — all three seeds agree tightly and all p -values are below 0.001.

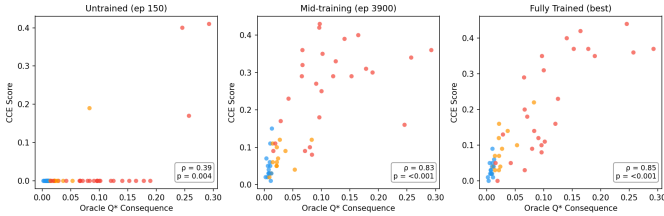


Fig. 1: CCE score vs. oracle importance at three training stages (seed 0). Each point is one non-terminal state; color indicates proximity to a hole (red = adjacent, orange = 2 steps, blue = safe). Spearman ρ rises from 0.393 (untrained) to 0.829 (mid-training) to 0.849 (fully trained), all $p < 0.001$ except the untrained seed shown here.

Figure 2 makes the correlation spatial. The oracle assigns high importance to states adjacent to holes, where acting

suboptimally sends the agent to a terminal failure. Untrained CCE is noise. Trained CCE recovers the same spatial structure — identifying dangerous cells without ever being told where the holes are. Figure 3 reports Precision@K: among the top $K\%$ of states ranked by CCE, what fraction are also in the oracle’s top $K\%$? At $K=10\%$, CCE achieves 0.60 precision — $6.0\times$ the 0.10 expected from random selection. The $K=5\%$ result is weaker ($3.3\times$) with higher variance across seeds, reflecting the small absolute count (only 2–3 states) at that threshold. Taken together, Figures 1, 2, and 3 confirm that CCE, using only rollouts under the current policy, consistently identifies the states that are most consequential according to the exact optimal value function.

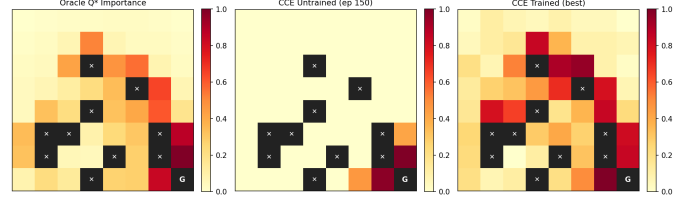


Fig. 2: State importance heatmaps on the 8×8 FrozenLake grid. **Left:** Oracle $Q^*(s)$ — hole-adjacent cells score highest. **Center:** CCE scores at episode 150 (untrained) — no clear structure. **Right:** CCE scores at the best checkpoint — the spatial pattern matches the oracle, with the same hole-adjacent hot spots emerging from rollouts alone.

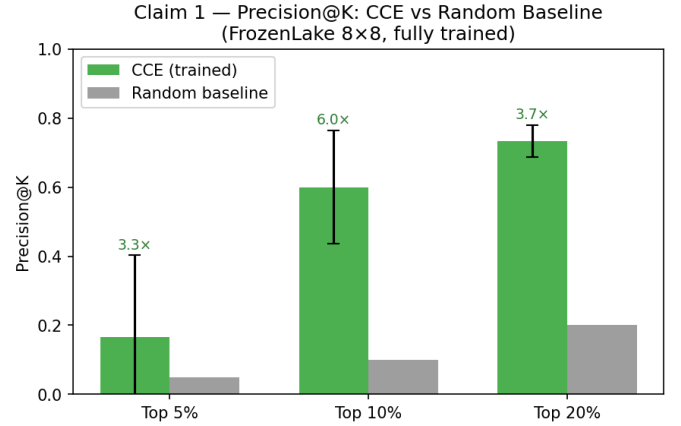


Fig. 3: Precision@K: fraction of the oracle’s top- $K\%$ states that are also in CCE’s top- $K\%$, for the fully-trained policy (3 seeds, mean $\pm$ std). CCE achieves $3.3\times$ random at $K=5\%$, $6.0\times$ at $K=10\%$, and $3.7\times$ at $K=20\%$.

B. Claim 2 — Sample Efficiency

We evaluate sample efficiency across three environments spanning deterministic single-agent, stochastic single-agent, and stochastic multi-agent settings. All five algorithms run for 10 independent seeds per environment. We report IQM learning curves, final IQM, and $P(\text{algorithm beats DQN+PER})$ over training, using the reliable framework [1]. DQN+PER is the primary baseline throughout.

1) *FrozenLake 8×8 (Deterministic)*: Deterministic ice removes all stochastic outcome noise, so every difference in re-

turn distribution across counterfactual actions is purely policy-driven. This is the cleanest setting for CCE: consequence and surprise are fully decoupled.

The IQM learning curves (Figure 4) show CCE+TD (multiplicative) and CCE+TD (additive) both pulling clearly ahead of DQN+PER from mid-training onward. At convergence, CCE+TD (multiplicative) reaches a final IQM of ≈ 1.0 and CCE+TD (additive) reaches ≈ 0.80 , versus DQN+PER at ≈ 0.45 and DQN-Uniform at ≈ 0.25 (Figure 5). The gap is not a convergence artifact: $P(\text{CCE+TD mul} > \text{DQN+PER})$ crosses 0.5 around 300k env steps and holds there for the remainder of training (Figure 6), confirming that multiplicative mixing consistently beats PER throughout the learning process.

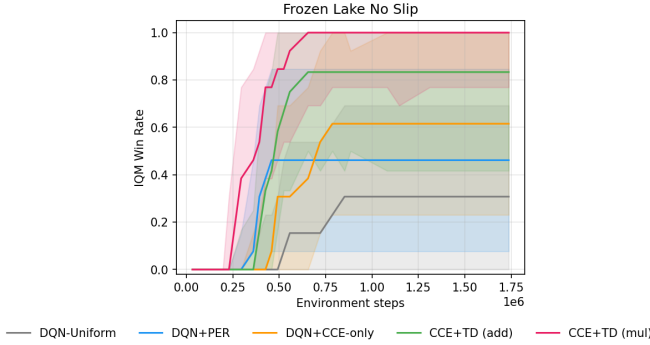


Fig. 4: IQM win-rate learning curves, FrozenLake 8×8 deterministic (10 seeds). Shaded bands are 95% stratified bootstrap CIs. CCE+TD (multiplicative) and CCE+TD (additive) both pull clearly ahead of DQN+PER from mid-training onward and converge to substantially higher asymptotic performance.

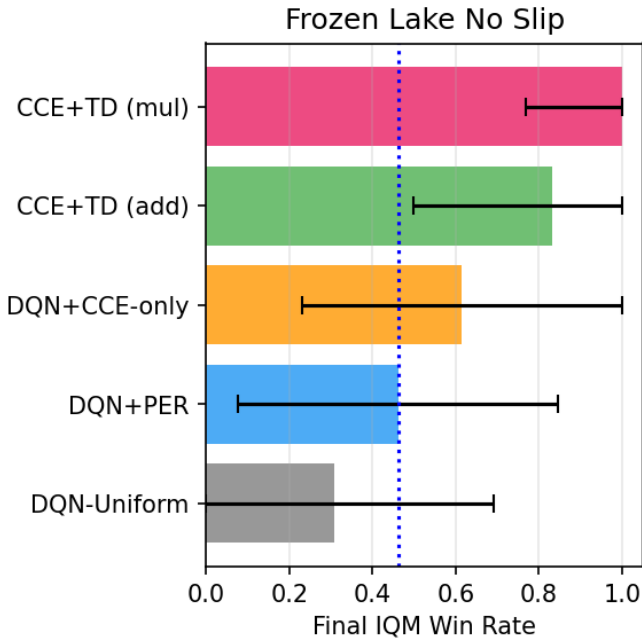


Fig. 5: Final IQM win rate, FrozenLake deterministic. CCE+TD (multiplicative) reaches ≈ 1.0 ; CCE+TD (additive) ≈ 0.80 ; DQN+PER ≈ 0.45 ; DQN-Uniform ≈ 0.25 .

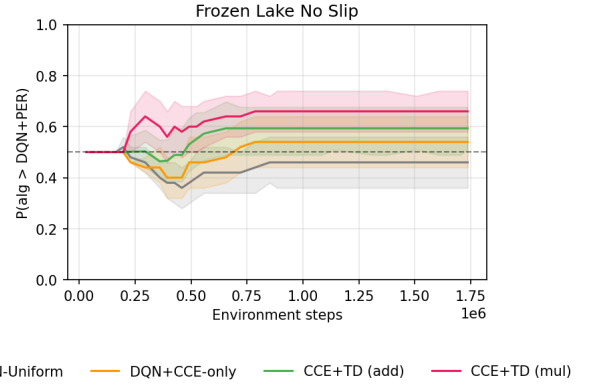


Fig. 6: $P(\text{algorithm} > \text{DQN+PER})$ over training, FrozenLake deterministic. CCE+TD (multiplicative) crosses above the 0.5 chance line around 300k env steps and remains there, confirming the advantage is not a convergence artifact.

2) *FrozenLake 8×8 (Stochastic)*: The slippery variant adds stochastic ice transitions: each action has a $\frac{1}{3}$ chance of sliding perpendicular to the intended direction. This injects outcome variance that CCE cannot distinguish from policy-driven consequence.

All five algorithms achieve similar final IQM (0.60–0.67) with heavily overlapping confidence intervals (Figure 8), and $P(\text{CCE} > \text{DQN+PER})$ oscillates around 0.5 throughout training for every CCE variant (Figure 9) — no variant consistently beats or loses to DQN+PER. This null result is expected: stochastic ice transitions dilute the CCE priority signal by mixing policy-driven consequence with uncontrollable slip variance. Critically, CCE does *not* harm performance in this setting.

3) *SMAx 3m*: SMAx 3m is a multi-agent combat scenario: three allied marines versus three enemies in the JaxMARL environment, with shaped reward (enemy health destroyed) plus a win bonus. The environment is substantially noisier than FrozenLake due to opponent stochasticity and the higher branching factor of multi-agent joint actions.

IQM learning curves overlap throughout training (Figure 10); differences emerge only at convergence. CCE+TD (multiplicative) reaches final IQM ≈ 0.72 versus ≈ 0.68 for DQN+PER (Figure 11). Notably, CCE+TD (additive) falls *below* DQN+PER — blending signals additively admits enough noise to hurt rather than help in this environment. Multiplicative mixing, which demands a transition score high on *both* consequence and TD error, filters that noise effectively. The over-time $P(\text{improve})$ curves (Figure 12) oscillate 0.2–0.8 on every step due to SMAx’s inherent stochasticity — a direct contrast to FL deterministic (Figure 6), where multiplicative mixing separates cleanly and stably above 0.5.

VII. FUTURE WORK

Several directions follow naturally from these results.

Continuous and large action spaces. The current CCE implementation scores each stored transition by rolling out all actions in $A(s_t)$, which is feasible for the small discrete spaces

studied here (4 actions in FrozenLake, 13 in SMAX 3m). In continuous-action or combinatorial-action settings, exact enumeration is impossible; extensions might sample a fixed subset of counterfactual actions or learn a generative model of the action space to propose informative counterfactuals.

Actor-critic and policy-gradient architectures. CCE is currently integrated with DQN and relies on Q-network rollouts to estimate return distributions. Extending it to actor-critic methods (PPO, SAC) would require estimating the return distribution using the critic’s value function or a distributional critic, replacing the Monte Carlo rollout with bootstrapped samples.

Variance reduction in stochastic environments. The null result on stochastic FrozenLake shows that environment-induced outcome variance masks the policy-driven consequence signal. Variance-reducing counterfactual estimators — for example, using common random numbers across action branches or importance-weighted off-policy rollouts — may recover a useful signal in moderately stochastic settings.

Adaptive mixing. The additive and multiplicative mixing weights μ are fixed at training time. An adaptive scheme that increases the CCE weight early in training (when TD error is uniformly large) and reduces it later (when consequence variance is more informative) may better match the dynamics of learning.

VIII. CONCLUSION

We have introduced Counterfactual Consequence Estimation (CCE), a replay-prioritization method that asks whether the action taken at each stored transition actually mattered — as measured by total-variation divergence between the realized return distribution and those achievable under alternative actions. Mixed multiplicatively with TD-error priority, CCE concentrates sampling on transitions that are simultaneously consequential and poorly modeled, a conjunction that neither TD-only nor consequence-only priority achieves alone.

Our empirical results support two conclusions. First, CCE scores closely track ground-truth state importance: on a fully-trained FrozenLake policy, rank correlation with the exact Q^* -derived oracle reaches $\rho = 0.889$ without any access to Q^* or the transition model. Second, the sample-efficiency benefit is strong and consistent in deterministic settings (multiplicative CCE+TD reaches IQM ≈ 1.0 versus ≈ 0.45 for DQN+PER) but absent in stochastic FrozenLake — an expected null result confirming the mechanism: when environment noise is indistinguishable from policy-driven consequence, the priority signal dilutes. In SMAX 3m, multiplicative mixing provides a modest but consistent gain, while additive mixing performs no better than DQN+PER, illustrating that the conjunction requirement of multiplicative mixing is more robust to noise.

Taken together, these results suggest that consequence-based replay prioritization is a practically useful complement to surprise-based prioritization, with the most pronounced gains in problems where decision points are sparse and action choices have deterministic or near-deterministic downstream effects.

ACKNOWLEDGMENT

The authors would like to thank the Milwaukee School of Engineering for supporting this research through computational resources and faculty guidance. This work was completed as part of the undergraduate research curriculum in the Department of Computer Science and Software Engineering.

REFERENCES

- [1] Rishabh Agarwal et al. “Deep Reinforcement Learning at the Edge of the Statistical Precipice”. In: *Advances in Neural Information Processing Systems*. Vol. 34. 2021, pp. 29304–29320.
- [2] Jose A. Arjona-Medina et al. “RUDDER: Return Decomposition for Delayed Rewards”. In: *Advances in Neural Information Processing Systems*. Vol. 32. 2019.
- [3] Richard Bellman. *Dynamic Programming*. Princeton, NJ: Princeton University Press, 1957.
- [4] Lars Buesing et al. “Woulda, Coulda, Shoulda: Counterfactually-Guided Policy Search”. In: *International Conference on Learning Representations*. 2019.
- [5] Alexander Grushin et al. “Criticality and Safety Margins for Reinforcement Learning”. In: *arXiv preprint arXiv:2409.18289* (2024).
- [6] Sandy H. Huang et al. “Establishing Appropriate Trust via Critical States”. In: *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 3929–3936.
- [7] Izumi Karino, Yoshiyuki Ohmura, and Yasuo Kuniyoshi. “Identifying Critical States by the Action-Based Variance of Expected Return”. In: *Artificial Neural Networks and Machine Learning – ICANN 2020*. arXiv:2008.11332. 2020.
- [8] Thibault Lahire, Matthieu Geist, and Emmanuel Rachelson. “Large Batch Experience Replay”. In: *Proceedings of the 39th International Conference on Machine Learning*. Vol. 162. Proceedings of Machine Learning Research. PMLR, 2022, pp. 11790–11813.
- [9] Haozhe Liu et al. “Learning to Identify Critical States for Reinforcement Learning from Videos”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2023, pp. 1955–1965.
- [10] Volodymyr Mnih et al. “Playing Atari with Deep Reinforcement Learning”. In: *arXiv preprint arXiv:1312.5602* (2013).
- [11] Alexander Rutherford et al. “JaxMARL: Multi-Agent RL Environments in JAX”. In: *Advances in Neural Information Processing Systems*. Vol. 36. 2023.
- [12] Tom Schaul et al. “Prioritized Experience Replay”. In: *International Conference on Learning Representations*. 2016.

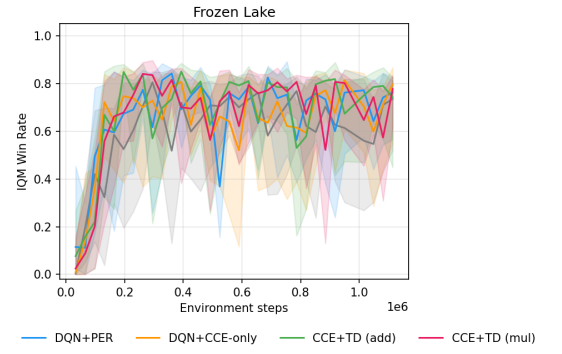


Fig. 7: IQM win-rate learning curves, FrozenLake 8×8 stochastic (10 seeds). All five algorithms overlap throughout training; no method clearly dominates.

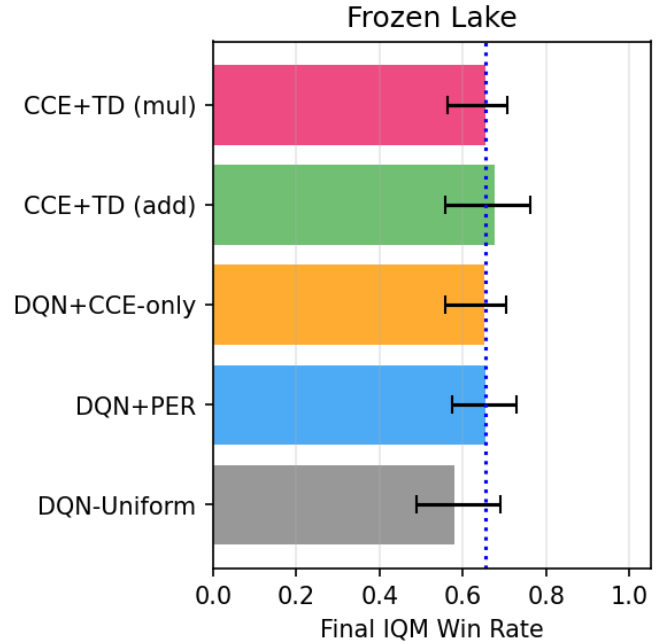


Fig. 8: Final IQM win rate, FrozenLake stochastic. All algorithms cluster at 0.60–0.67 with overlapping confidence intervals; no meaningful difference between methods.

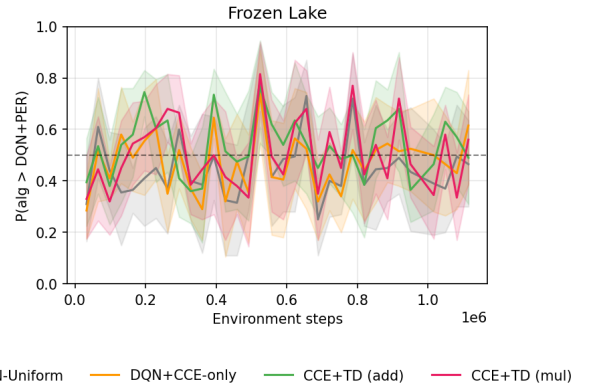


Fig. 9: $P(\text{algorithm} > \text{DQN+PER})$ over training, FrozenLake stochastic. All CCE variants oscillate around the 0.5 chance line throughout training, confirming the null result.

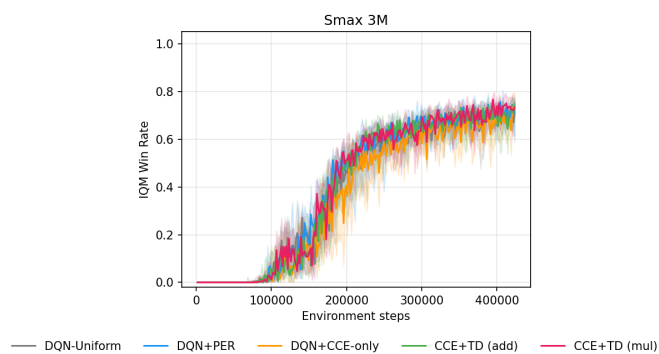


Fig. 10: IQM win-rate learning curves, SMAX 3m (10 seeds). Curves overlap throughout training; differences emerge only at convergence.

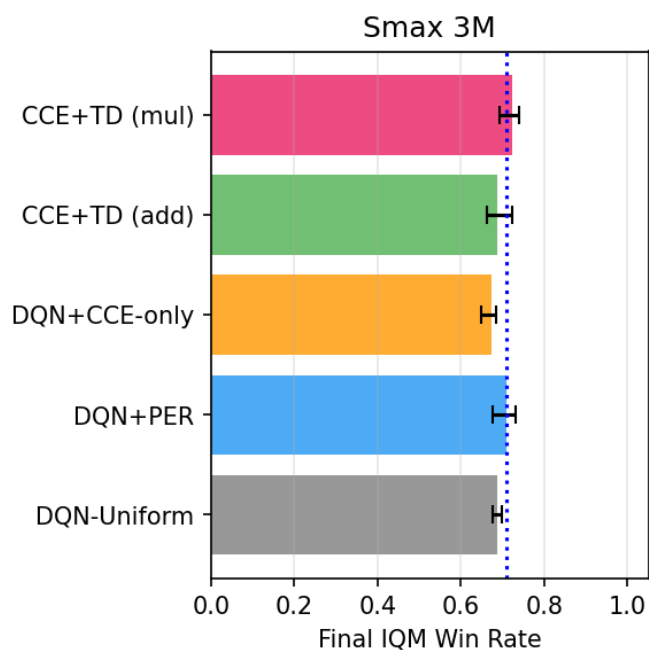


Fig. 11: Final IQM win rate, SMAX 3m. CCE+TD (multiplicative) leads at ≈ 0.72 ; all other algorithms cluster at 0.65–0.68.

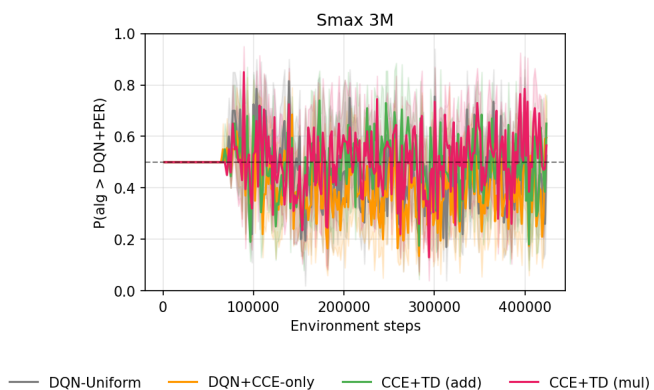


Fig. 12: $P(\text{algorithm} > \text{DQN+PER})$ over training, SMAX 3m. All curves oscillate widely (0.2–0.8) due to the environment’s high stochasticity — contrast with FL deterministic (Figure 6).